Supabase Auth Migration Security Impact Analysis

How the application's security changed, the trade-offs, and future costs

Prefects Discipline · Discipline Management Dashboard

17 August 2026

SUMMARY

This report covers the move from the previous custom authentication (client-side bcrypt comparison, localStorage sessions, no Row-Level Security) to real Supabase Auth: email/password accounts, server-verified httpOnly-cookie sessions, server-side admin routes, and Row-Level Security on every table. It explains what got safer, what was given up, and what the new implementation is likely to cost going forward.

1. What changed

Aspect	Before	After
Password storage	bcrypt hash in the users table, compared in the browser (bcryptjs)	Managed by Supabase Auth in auth.users — hashed server-side, never seen by the browser
Session	Plain {id, username, role} JSON in localStorage — editable by anyone with devtools	httpOnly cookies via @supabase/ssr; access + refresh tokens managed by Supabase
Route protection	Client-side redirect after hydration (a flash of the login gate)	proxy.ts refreshes the session and redirects unauthenticated requests server-side
Role source of truth	localStorage user object (re-verified against the DB on mount)	Fetched from the users table by auth_id on every session start — role changes apply immediately
User management	Client-side inserts/updates with the anon key; any logged-in admin could call them	Server-side /api/users routes using the service-role key; caller verified as superuser on the server
Destructive actions	Password re-entered and bcrypt-compared in the browser	Re-verified with a real signInWithPassword call against the user's account
Row-Level Security	None — the public anon key could read/write/delete every table	Enabled on all 10 tables with role-aware policies (authenticated read; admin/superuser write; superuser-only user management)
Config	Supabase URL + anon key hardcoded in the client bundle	Env-driven (NEXT_PUBLIC_*); service-role key server-only

2. Security holes closed

Three issues previously flagged as critical in the project's security review are now fixed:

- **Closed: P0 —**No RLS / anon-key access. Anyone who fetched the public key from the bundle could call the Supabase REST API directly and read, write, or delete every table without logging in. RLS now denies unauthenticated access entirely, and role-aware policies restrict writes to admins and superusers.
- **Closed: P0 —**Role impersonation via localStorage. Editing localStorage to role: "admin" granted the admin UI. Sessions are now httpOnly cookies, and the role always comes from the database for the authenticated auth_id.
- **Closed: P0 —**Auth was only a UI gate. There was no server-side boundary — the login screen merely hid buttons. There is now a real access boundary: proxy.ts rejects unauthenticated page loads, API routes verify the caller server-side, and RLS enforces the same role matrix at the database.
- **Closed: P2 —**Client-side bcrypt. Password hashes were shipped to and compared in the browser. Passwords are now handled entirely by Supabase Auth on the server; bcryptjs was removed from the project.

- **Closed: P1 —** Hardcoded credentials in the bundle. The Supabase URL/key now come from environment variables, and the service-role key (which bypasses RLS) exists only server-side.
- **Not addressed: P1 —** Forged issuer names on records (strikes/comments). Not fixed — record tables still store free-text issuer names. Attribution to the real logged-in user is now possible (auth.uid()) but not yet implemented.
- **Not addressed: P1 —** No audit trail. Deletes are still permanent and untracked. With authenticated sessions in place, an audit_log table + triggers is now straightforward — but it has not been built yet.

3. Pros

- Sessions cannot be forged or stolen from localStorage — httpOnly cookies, token refresh and rotation handled by Supabase.
- Passwords never touch the browser or the application code; hashing, salting, and storage are managed by Supabase Auth.
- Row-Level Security is the enforcement layer the app always lacked: even a malicious client can no longer exceed its role at the database.
- Server-side admin routes mean user creation, role changes, and deletion are authorized on the server, not just hidden in the UI; self-deletion / last-superuser guards are enforced there too.
- Destructive actions (delete record, clear strikes, CSV upload) are re-verified with a real authentication call — stronger than the old browser-side bcrypt compare.
- Role changes by a superuser take effect on the affected user's next page load (role is re-fetched from the DB, not cached forever).
- Users can change their own password without a superuser; sessions stay logged in across restarts; multiple tabs stay in sync via onAuthStateChange.
- Managed infrastructure: no server to run, patch, or secure for authentication — Supabase owns that surface.
- The login page, role hierarchy (superuser > admin > view-only), and every role-gated feature behave exactly as before the migration.

4. Cons

- Usernames are now tied to synthetic emails (username@prefects.local). No real email addresses exist, so email confirmation, 'forgot password', and email invites are not usable without switching to real emails later.
- Password policy floor: Supabase Auth always requires a password and enforces a minimum length (default 6). The previous 'any non-empty password' behavior is impossible to fully preserve.
- Usernames are immutable after creation (they define the auth email). Previously a superuser could rename a user.
- Two sources of truth — auth.users and the users table — must be kept in sync: a user is two rows, and delete/create must touch both (the code does this, but it is more state to reason about).
- RLS adds a new maintenance surface: every new table, column, or query needs a policy review; mistakes default to 'denied', which can silently break features.
- Legacy accounts could not be migrated (bcrypt hashes are one-way) — existing users had to be recreated with fresh passwords, and the app is unusable until the migration SQL has been applied.
- proxy.ts adds an authentication round-trip on every page request (session refresh/validation), a small latency cost at each navigation.
- The service-role key is now a secret that must be managed (env var in deployment, rotation discipline). Leaking it would bypass all RLS.
- Tight coupling to a third-party auth service: login availability now depends on Supabase Auth's uptime, and the project depends on @supabase/ssr staying compatible with this custom Next.js version (proxy.ts instead of middleware).

5. Potential future costs

Item	Estimate / cost	Notes
Supabase Auth usage	Free at this scale	Free tier covers 50,000 monthly active users — far beyond a school's staff. Pricing only matters if the app grows to tens of thousands of users.
RLS policy maintenance	"H 0 . 5 – 1 d a schema change	Every new table/column needs policies. One-time cost each time; ongoing reviews should be part of code review.
Service-role key management	"H m i n u t e s rotation	Rotate on staff change or suspected leak; store only in server env vars, never client-side.
Moving to real emails later	1–2 days + email provider	Required before email confirmation / password reset / invites become possible. Supabase's built-in email is free at school volume; a custom SMTP provider is optional for better deliverability.
Password reset / confirmation flow	0.5–1 day (once real emails exist)	Supabase supports both out of the box — mostly configuration plus small UI additions.
Multi-factor authentication	"H 0 . 5 d a y config	Supabase supports TOTP MFA; enabling it is configuration plus a small enrollment screen.
Audit trail (open P1)	1–2 days	audit_log table + Postgres triggers using auth.uid() — now feasible since sessions are real. Recommended next step.
Rate limiting / lockout	"H 0 . 5 d a y config	Supabase offers built-in rate limiting and the option to require email confirmation before sign-in to slow brute force.
Dependency upkeep	Occasional	Keep @supabase/ssr and supabase-js updated; verify proxy.ts compatibility on each Next.js major upgrade.
Operational dependency	0 (money) / small (risk)	No self-hosted infrastructure was added — but logins depend on Supabase Auth uptime. No new ops burden.

6. Summary

The migration converted the application's authentication from a cosmetic, client-side gate into a real security boundary. The three critical findings of the earlier security review — no RLS, forgeable localStorage sessions, and UI-only role enforcement — are closed. Passwords are now handled entirely by a managed auth service, destructive actions are re-verified with real credentials, and user management is authorized server-side.

The trade-offs are modest and mostly ergonomic: synthetic emails rule out email-based flows for now, usernames are immutable, and Supabase's password floor replaces the old 'any non-empty' rule. The ongoing costs are small — policy maintenance when the schema changes, secret hygiene for the service-role key, and dependency upkeep. At this application's scale, the new implementation is cheaper to operate than the old one was to secure.

RECOMMENDATION

Verdict: the new authentication is a clear security improvement with low ongoing cost. Recommended next steps — in order of value — are an audit trail (now easy with real authenticated sessions), issuer attribution on records (auto-fill from the logged-in user), and configuring Supabase's rate limiting. Real email addresses are only worth switching to if staff actually need password reset or email invites.